

Tech Assessment 01: Async Summarization Service

Overview

Time budget: 4-5 hours. Please do not spend more. We would rather see a smaller, well-made thing plus a note about what you would do next than a large unfinished one. Finishing every part is not expected of anyone. AI coding assistants are allowed and expected.

Language, framework and datastore are entirely your choice. Write idiomatic code in a stack you know well. We are not checking whether you picked ours.

The problem

Build a service that accepts a block of text, summarizes it asynchronously by calling a **model provider**, and lets the caller poll for the result.

The catch, and the actual point of the exercise: the provider is slow, unreliable, rate-limited, and charges per token. Your service sits between users and that provider and has to behave sensibly anyway.

The model provider

You can choose any model provider as you wish. Implement it exactly to this contract:

Behavior	Specification
Latency	Uniformly random between 1000 ms and 3000 ms per call
Transient failure	~15% of calls return HTTP 500
Concurrency limit	At most 2 calls may be in flight at once. A call beyond that returns HTTP 429 with a <code>Retry-After</code> header of 1-3 seconds
Invalid input	Text longer than 10,000 characters , or empty, returns HTTP 400 with an error code such as <code>invalid_input</code>
Success response	<code>{ "summary": "...", "input_tokens": N, "output_tokens": M }</code>

Token counting	<code>input_tokens = ceil(len(text) / 4),</code> <code>output_tokens = ceil(input_tokens * 0.2)</code>
Pricing	\$0.003 per 1K input tokens, \$0.015 per 1K output tokens
Determinism	Seeded from a <code>SEED</code> env var, so a given run is reproducible

The summary text itself can be trivial (first two sentences, a truncation, anything). **We are not assessing summary quality** — only how your service behaves around the call.

Keep the stub behind an interface so it can be swapped or faked in tests. Whether it runs in-process or as a separate HTTP service is up to you; treat it as if it were remote either way.

API

`POST /jobs`

```
1 { "text": "...", "callback_url": "https://..." }
2
```

`callback_url` is optional. Returns **202** with a job id.

`GET /jobs/{id}`

Returns the job's status, plus:

- the summary, once it succeeded
- the error detail, if it did not
- how many attempts were made
- input tokens, output tokens, and cost accumulated by this job

`GET /healthz`

Liveness/readiness for the service.

Job states

```
1 queued -> running -> succeeded | failed | dead
2
```

- `succeeded` — the provider returned a summary.
- `failed` — the job cannot succeed no matter how many times it is retried. The provider rejected the input itself.

- `dead` — the job kept hitting errors that *might* have succeeded on a later attempt, and ran out of retry budget.

Deciding which provider errors belong in which bucket is part of the exercise. **Say in your README how you classified them and why.**

Scope

Core is required. Everything after it is optional. Work down the list as far as your time allows, then stop and write down what you skipped. Leaving an extension undone is a normal outcome and is not counted against you; claiming something works when it does not is a different matter.

Core — required

- `POST /jobs`, `GET /jobs/{id}`, `GET /healthz` working end to end
- Job state survives a process restart
- A README with exact commands to run it and to run the tests
- Tests — some. Your call which.

Extension A — reliability

- The worker runs separately from the API request path
- Retries with backoff, honoring `Retry-After` on 429
- `docker compose up` brings up the whole thing
- Tests that exercise failure paths, not only the happy path

Extension B — production readiness

- An idempotency key on `POST /jobs`, so a retried submit does not create a second job
- Retryable vs terminal error classification, with exhausted jobs dead-lettered rather than silently dropped
- Multiple workers can run at once without any job being processed twice
- Backpressure or rate limiting on your side of the provider's limit
- Graceful shutdown: no job is lost or stuck mid-flight when a worker stops
- Health and metrics worth putting on a dashboard
- Token and cost accounting, per job and in total

Extension C — the write-up

A `DESIGN.md`, one to two pages, covering whichever of these you have opinions about:

- The trade-offs you made and what you gave up
- How the system fails, and what happens to in-flight work when it does

- Whether you deliver at-least-once or exactly-once, and how you know
 - What you would do about a job that poisons a worker every time it runs
 - One noisy tenant submitting 10,000 jobs — what happens to everyone else
 - Where this breaks at 100x the load, and what you would change first
 - Controlling provider spend
 - What you would build next if you have more weeks
-

What to send us

A git repository — a link or a zip, either is fine — containing:

- your code
- `README.md` : how to run it, how to run the tests, what you chose to skip and why, anything you would want a reviewer to know
- `DESIGN.md` if you did Extension C

Please keep your commit history. We read it, and a series of honest commits tells us more than one squashed "initial commit".

Not needed

- Any frontend or UI, this is optional, you can still do if you want.
- Authentication, user accounts, or multi-tenancy plumbing
- A real LLM, real prompts, or any evaluation of summary quality
- Cloud deployment, Terraform, or Kubernetes manifests
- Perfection. We would rather see a clear "I ran out of time here, and this is what I would have done" than filler.

Afterwards

We will read the code and then walk through it with you for about an hour — how it works, why you made the calls you made, and what you would change. It is a conversation about your work, not a quiz. **Bring the questions you had while building it;** the ones you could not resolve are usually the most interesting part.